👗 Urban-Edge Stylist

AI Fashion Recommendation System (Capstone Project)

---

1. Introduction

Urban-Edge Stylist is an intelligent AI-based fashion recommendation system that helps users discover outfits based on their preferred **style vibe** such as minimalist, edgy, casual, sporty, or formal.

Unlike traditional systems that rely on keyword matching, this system uses **semantic understanding** to interpret the meaning behind user queries. For example, if a user searches for *"edgy streetwear outfit"*, the system retrieves items that match the *style intent*, not just exact words.

The system follows a **Retrieval-Augmented Generation (RAG)** approach:

- Retrieval using vector similarity (FAISS)
- Generation using structured output and Gemini API

This combination ensures both **accuracy and human-like responses**, making the system more intelligent and user-friendly.

---

2. Objective

The main objectives of this project are:

- To recommend outfits based on **style preferences**
- To implement **semantic search** using embeddings
- To enrich data using **metadata tagging** (style, category)
- To design an **intent routing system** for query understanding
- To enable **fast retrieval** using FAISS
- To integrate **Gemini API for natural language generation**
- To build a **clean and interactive user interface**
- To ensure the system is **robust, safe, and efficient**

---

3. Methodology

The system follows a structured pipeline:

🔹 **Step 1: User Query Input**

The user enters a query such as:

"minimalist outfit" or "latest fashion trends"

🔹 **Step 2: Safety Validation**

The system checks:

- Unsafe or offensive words
- Very short or invalid queries

Invalid inputs are rejected safely.

🔹 **Step 3: Intent Routing**

The query is classified into:

- **Recommendation**
- **Trend**
- **General**

This determines how the system responds.

🔹 **Step 4: Query Embedding**

The query is converted into a vector using a **sentence-transformer model (all-MiniLM-L6-v2)**.

🔹 **Step 5: FAISS Vector Search**

The system retrieves top-k similar fashion items using FAISS based on semantic similarity.

🔹 **Step 6: Metadata Attachment**

Each retrieved item contains:

- Style vibe
- Category
- Description

🔹 **Step 7: Response Generation (Enhanced with Gemini)**

- For **recommendation queries** → structured output from FAISS retrieval

- For **trend/general queries** → Gemini API generates human-like responses

This ensures both:
✓ Accuracy (retrieval)
✓ Readability (generation)

---

4. System Architecture

User Query

↓

Safety Check

↓

Intent Router

↓

Embedding Model

↓

FAISS Vector Database

↓

Metadata Retrieval

↓

Gemini API (for generation)

↓

Response Output

↓

Streamlit UI

Key Components:

- **Embedding Model** → Converts text into vectors
- **FAISS Database** → Performs similarity search
- **Metadata Layer** → Adds structured information
- **Intent Router** → Controls logic flow
- **Gemini API** → Generates natural language responses
- **UI (Streamlit)** → Displays results

---

5. Technologies Used

Technology	Purpose
Python	Core development
FAISS	Fast similarity search
Sentence Transformers	Text embeddings
NumPy	Numerical operations
JSON	Data storage
Streamlit	User interface
Gemini API	Natural language generation

---

6. Use of Gemini API (LLM Integration)

The system integrates the **Gemini Text Generation API** to enhance response quality.

🔹 **Purpose**

Gemini is used for:

- Generating **fashion trends**
- Answering **general queries**
- Enhancing recommendation descriptions (optional)

API Authentication

- Uses GEMINI_API_KEY
- Stored securely in .env
- Loaded in main.py

API Endpoint

https://generativelanguage.googleapis.com/v1beta2/models/{model}:generateText?key={api_key}

Working Flow

1. User query is received
2. Intent is identified
3. For trend/general → request sent to Gemini
4. Gemini returns generated text
5. Output is displayed

Example

Query:

latest fashion trends

Output:

Oversized silhouettes, neutral tones, and minimal layering are trending in modern fashion.

Importance

Component	Role
FAISS	Retrieval
Metadata	Structuring
Gemini	Natural response generation

🔹 **Fallback**

If API fails:

- System uses predefined responses
- Core system still works

7. Features

- 🔍 Semantic search (meaning-based)
- 🎯 Style vibe recommendations
- 🗂️ Metadata tagging
- ⚡ Fast FAISS retrieval
- 🧠 Intent-based routing
- 🤖 Gemini-powered responses
- 🎨 Interactive UI
- 🛡️ Safety validation

---

8. Project Structure

```
capstoneproject/
│
├── Day11_Task/   # FAISS + dataset
├── Day12_Task/   # Logic + routing
├── Day13_Task/   # Testing
├── Day14_Task/   # Optimization
├── Day15_Task/   # UI
│
└── README.md
```

---

9. Testing & Evaluation

The system was tested using:

- **Normal Testing**
  - Verified recommendation accuracy
- **Red-Team Testing**
  - Adversarial queries
  - Random inputs
- **Safety Testing**
  - Blocked unsafe queries
- **Error Handling**
  - No crashes on invalid input

---

10. Performance Optimization

- FAISS enables fast retrieval
- Precomputed embeddings improve speed
- Top-k search reduces overhead
- Optimized prompts reduce API cost

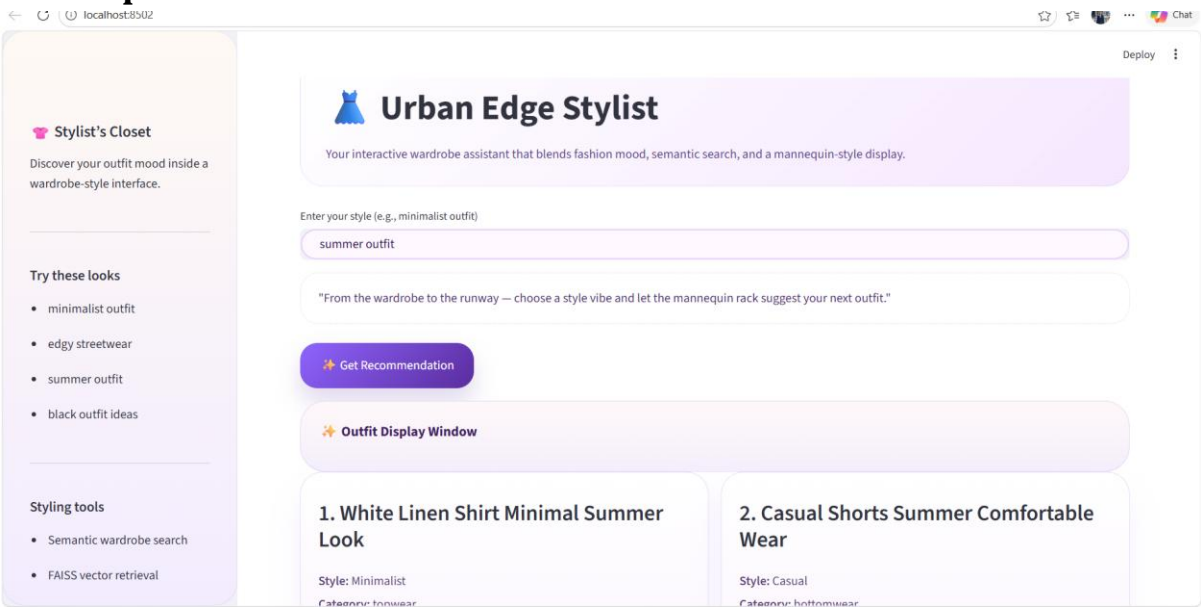
---

11. Challenges and Solutions

Challenge	Solution
Import errors	Fixed using sys.path
File path issues	Used os.path
Invalid inputs	Safety validation
Output quality	Gemini integration
Data structure	JSON metadata

---

12. Output



---

13. Conclusion

Urban-Edge Stylist successfully demonstrates a **hybrid AI system combining retrieval and generation**.

By integrating:

- Semantic embeddings
- FAISS vector search
- Metadata-driven design
- Gemini API

the system delivers:

✓ Accurate recommendations
✓ Fast performance
✓ Human-like responses
✓ Robust architecture

This project showcases a **real-world AI application** with both engineering and intelligence, making it highly valuable for modern recommendation systems.